

GOAI | 旅行供应链异常多 Agent 自主闭环

1. 场景与价值 | 供应链异常为什么难以自动化

供应链履约是一个让很多领域、公司企业都头疼不已的问题。供应链履约异常往往跨分销商、平台、供应商和服务团队。信息、权限和责任分散，越权处置会带来错单、资金或服务风险。以酒店分销场景为例，当客人发现自己在 OTA 预订的订单消失后，可能会带着怨气和不满找到平台客服，这么问道：“为什么我的订单查不到了？”，“为什么半小时还没回复我？”。客人只知道ta的应用页面中的订单人间蒸发了，但客人不知道这背后可能涉及订单归属、客户身份、系统权限，以及 OTA、分销商与酒店之间的履约确认。前台的一次查询，往往需要跨角色协作才能给出可信答案。

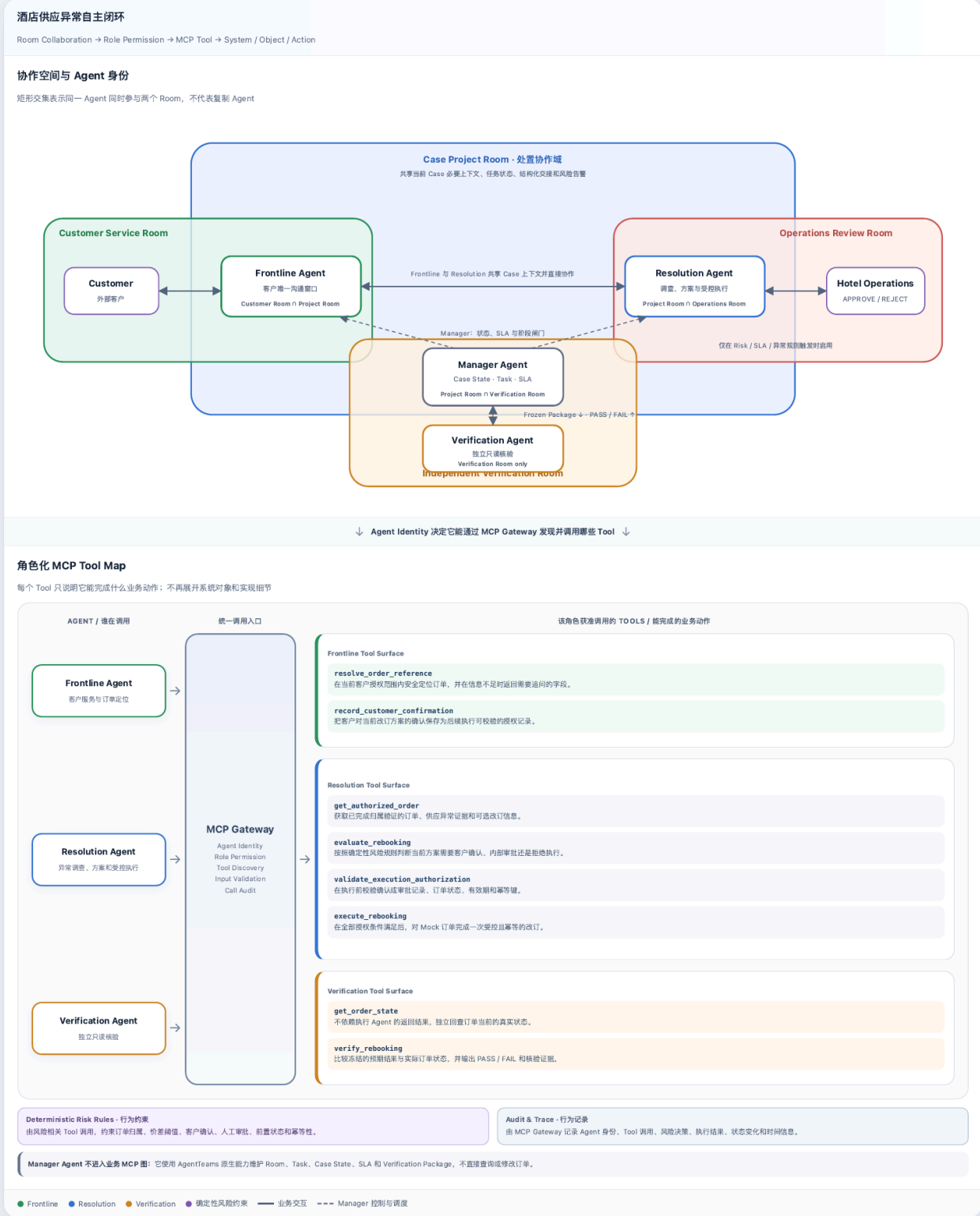
本方案将聚焦在旅行分销场景中，客户预订酒店后发现平台上查无订单，随即向平台求助。平台需要在不暴露其他订单信息的前提下，协调订单、供应与售后环节，并给出可执行的处理方案，直到确认问题已真正解决，实现智能客服自主闭环。

2. 方案设计 | 从业务分工到受控协作

2.1 整体架构 | Plan—Execute—Evaluate 框架下的 Multi-Agent 协作机制

GOAI 多 Agent 协作与 MCP Tool 架构 · V7

上半部分回答“谁在哪里协作”；下半部分只回答“每个 Agent 通过 MCP 获准调用哪些 Tool，以及这些 Tool 能完成什么业务动作”。



酒店分销的客户服务，表面上是统一的查询或对话入口，内部却需要客服与售后履约团队共同完成。客服承接诉求、补充信息并维持对客沟通；售后核对订单与供应状态，调查异常、生成处置方案并完成受控执行。客户看到的一次“查、退、改”，背后实际是一组跨角色、跨系统的协作任务。

本方案先在抽象层将多 Agent 的关系归纳为 **Plan – Execute – Evaluate**：

抽象责任	Agent Identity	核心职责	边界
Plan	主控 Agent (Manager)	将客户问题转化为 Case 目标，分析、拆解任务、调度/协调一或多个子Agent	不查询或修改业务订单，不替代结果核验
Execute	客服 Agent (Frontline)	承接客户对话、了解用户意图，获取查询订单的必要信息、安全定位订单并记录客户确认	不调查供应异常，不修改订单
Execute	售后 Agent (Resolution)	根据前台发来的需求，对业务数据库进行“增删改查”操作	不判定客户身份，不自行审批高风险动作，不核验自身结果
Evaluate	评估 Agent (Verification)	依据冻结输入独立回读实际订单，输出 PASS / FAIL 与核验依据	不加入前期处置讨论，不修改方案或订单

主图：GOAI 多 Agent 协作与 MCP Tool 架构 · V7。

2.2 Room 协作机制 | 业务动作如何在不同空间中流转

Plan – Execute – Evaluate 在抽象层定义了 Agent 的责任关系。Room 则承接它们的具体协作，每个 Agent 仍是同一个身份，但可以根据任务加入不同 Room；每个 Room 只保留完成当前业务动作所需的上下文。

Room	参与者	承接的业务动作
Customer Service Room	客户、客服 Agent	接收诉求、追问必要信息、发布处置方案、记录客户确认并通知最终结果
Case Project Room	主控 Agent、客服 Agent、售后 Agent	围绕同一 Case 拆解任务、共享必要上下文、直接交接进展并维护全局状态
Operations Review Room	售后 Agent、人工运营	在风险规则触发时提交方案和风险依据，由人工运营作出 APPROVE / REJECT
Independent Verification Room	主控 Agent、评估 Agent	传入冻结后的 Verification Package，独立回读实际订单并返回 PASS / FAIL

多 Room 协作包含三种关系：

- 同级协作：客服与售后围绕同一需求共享必要上下文；
- 总控调度：主控 Agent 掌握全局进度、SLA，并协调各子 Agent；
- 分域隔离：客户沟通、人工审批与结果核验分别置于独立 Room，避免信息无差别共享。

多 Room 协作，其实是 Agent 身份边界在业务场景中的延伸。客服与售后需要频繁配合，因此被放进同一个“协作会议室”，共享当前问题所需的部分上下文；主控 Agent 则与每个子 Agent 保留独立聊天 Room，会议室里只谈当前任务，不是什么都聊，这其实和人类的真实协同是一致的。评估 Agent 也被单独放在一个 Room：主控按需调度，

并只同步必要信息，让它站在“旁观者”视角，不受冗余上下文干扰，给出中立判断。这样的设计兼顾了上下协调、同级协作和客观评估。

2.3 Skill 与工具集成 | 身份边界如何落实为工具权限

在明确身份与上下文边界后，我们根据Agent的角色定位设计对应的用 Skill。再通过 MCP Gateway 向不同身份暴露不同的 Tool 。Agent Identity 决定“谁来做、边界在哪”，Skill 定义“这件事怎么做”，MCP Tool 则负责把动作真正执行下去。Agent 执行 Skill 时，只能通过 MCP Gateway 调用其身份获准的 Tool。这样一来，MCP Gateway 不只是统一调用入口，还负责身份识别、角色权限、Tool Discovery、输入校验和调用审计。客服 Agent 不能发现订单写入工具，评估 Agent 只能发现只读核验工具，主控 Agent 则不挂载业务 Tool。

Skill	身份与调用条件	关键 MCP Tool	输出与失败处理
identify-hotel-order	客服 Agent (Frontline)；客户报障且订单尚未唯一定位	resolve_order_reference 、 record_customer_confirmation	输出已授权订单引用或最小补充信息请求；多候选时不返回订单详情
investigate-hotel-supply-exception	售后 Agent (Resolution)；已获得有效订单引用	get_authorized_order 、 evaluate_rebooking 、 validate_execution_authorization 、 execute_rebooking	输出异常依据、处理方案、风险结论和受控执行记录；未确认、未审批或前置状态失效时拒绝执行
verify-hotel-rebooking	评估 Agent (Verification)；已收到完整且冻结的 Verification Package	get_order_state 、 verify_rebooking	输出 PASS / FAIL 与核验依据；只读回查实际订单，不采信执行 Agent 自报结果

2.4 可信处置闭环 | 审批、核验与留痕

2.4.1 受控运行 | 人工审批与独立核验

当方案命中人工审批规则时（本方案以 800 元价差为例），售后 Agent 会将当前问题、风险原因和建议方案送交运营人员，由其作出批准或拒绝。

审批期间，系统暂停自动处理，客户侧显示“方案审核中”，原订单保持不变。人工批准也不等于立即执行：只有运营批准与客户对同一方案的确认同时有效，售后 Agent 才能执行。完成后，评估 Agent 会重新读取订单并独立判断结果，不能由执行方自行证明成功。

2.4.2 过程可追溯 | 留痕与记录存储

系统会按服务问题记录角色交接、方案版本、客户确认、人工决定、工具调用、执行前后状态和核验结果，但不保存 Agent 的隐藏推理。问题关闭后，记录不会被删除：当前版本将问题状态、协作空间、处理过程和核验结果保存在本地，供演示与复核使用。数据库和对象存储将在后续建设。

3. 后续计划

目前方案构建的系统已经能保存每次处理的状态和过程记录，但这些内容主要用于复核，还不能被再次利用。后续建设将集中在业务知识和风险决策两项能力。

3.1 业务数据与知识库

订单、库存、履约状态和处理记录属于实时业务数据，由业务数据库管理，并通过工具按权限读取。标准作业流程、供应商处理手册和经过审核的历史处置经验将进入检索增强知识库（RAG）。处理问题时，Agent 先取得当前业务事实，再按业务类型、供应商、权限和有效期检索相关经验，检索结果保留来源。处理结束后，只有经过核验和筛选的处理摘要才会进入知识库，避免错误经验被重复使用。

3.2 可插拔风险规则引擎

风险判断将从 Agent 指令中拆出，形成独立、可版本管理的规则引擎。

- L1 为查询和解释，由 Agent 自主完成；
- L2 为低风险、可回退的变更，取得客户确认后执行；
- L3 为高金额或不可逆操作，需要客户确认和人工审批；
- L4 为合规冲突或禁止自动化的操作，停止执行并转交人工。

引擎统一返回风险等级、触发原因、允许动作和所需授权；增加新业务时，只需接入新的规则包，不必逐一改写 Agent。